



Read Chapters 2, 4 and 5

Unit 1_03

Java Fundamentals

Decision Structures

Outline

- Relational Operators
- if statement
- Conditional Operator
- switch statement

Java Decision Structures



Decision Making: Equality and Relational Operators

- **Condition :**
  - An expression that can be true or false.
- **if selection statement :**
  - Allows a program to make a decision based on a condition’s value.
- **Equality operators :**
  - (== and !=)
- **Relational operators :**
  - (>, <, >= and <=)

Decision Making: Equality and Relational Operators

Algebraic operator	Java equality or relational operator	Sample Java condition	Meaning of Java condition
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y

Decision Making: Equality and Relational Operators

```
Scanner input = new Scanner(System.in);
int a, b;
// prompt for input
System.out.print("Enter two integer values a and b: ");
a = input.nextInt(); // read first number
b = input.nextInt(); // read second number
if (a == b) {
    System.out.printf("%d and %d are equal.%n", a, b);
}
if (a != b) {
     System.out.printf("%d and %d are NOT equal. %n", a, b);
}
if (a > b) {
     System.out.printf("%d > %d %n", a, b);
}
```

```
Enter two integer values a and b: 7
76
7 and 76 are NOT equal.
```

```
Enter two integer values a and b: 18
3
18 and 3 are NOT equal.
18 > 3
```

```
Enter two integer values a and b: 15
15
15 and 15 are equal.
```

Selection Statements in Java

if statement:

- Performs an action, if a condition is true; skips it, if false.
- Single-selection statement—selects or ignores a single action (or group of actions).

if…else statement:

- Performs an action if a condition is true and performs a different action (or groups of actions) if the condition is false.

switch statement

- Performs one of several actions (or groups of actions), based on the value of an expression.

if Statement

- Single option

```
if (boolean-expression) {
  statement1;
  ...
  statementN;
}
```

- Two options

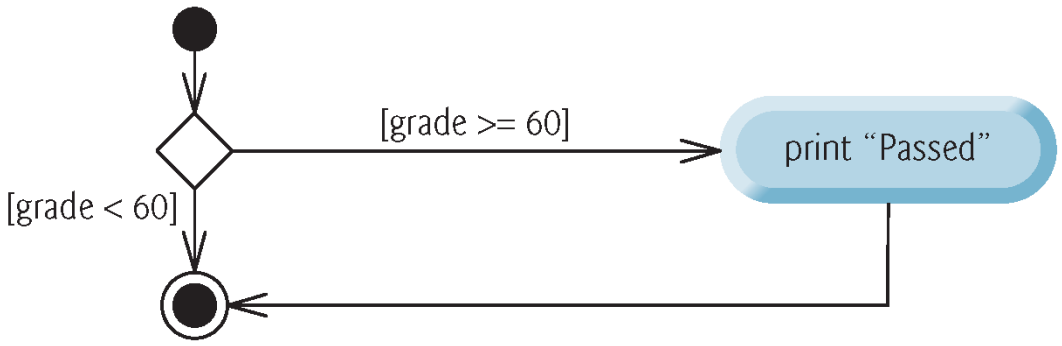
```
if (boolean-expression) {
  ...
} else {
  ...
}
```

- Multiple options

```
if (boolean-expression) {
  ...
} else if (boolean-expression) {
  ...
} else if (boolean-expression) {
  ...
} else {
  ...
}
```

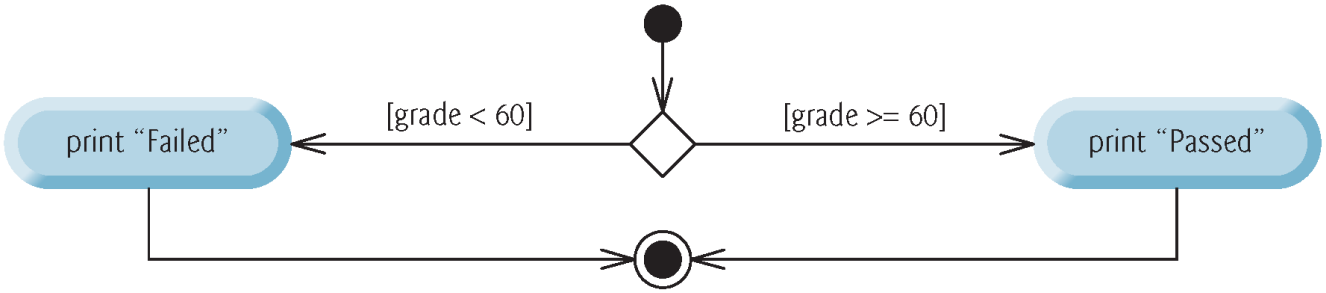
if Single-Selection Statement

```
public static void main(String[] args) {
    double studentGrade =70;
    if (studentGrade >= 60) {
        System.out.println("Passed");
    }
}
```



if…else Double-Selection Statement

```
public static void main(String[] args) {
    double studentGrade =70;
    if (studentGrade >= 60) {
        System.out.println("Passed");
    }else {
        System.out.println("Failed");
    }
}
```



Using if…else

```
import java.util.Scanner;

public class OddEven {
 public static void main(String arg[])
 {
    Scanner input = new Scanner( System.in );
    int x;
    System.out.print( "Enter an integer: " );
     x = input.nextInt();

     if (x%2==0)
        System.out.printf("%d %s",x,"is even number");
     else
        System.out.printf("%d %s",x,"is odd number");
 }
}
```

Nested if…else Statements

Nested if…else Statements:

- A program can test multiple cases by placing if…else statements inside other if…else statements to create ***nested if…else*** *statements*.

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an ineger: ");
int i = input.nextInt();
if (i == 10)
{
     // First if statement
     if (i < 15)
     {
          System.out.println("i is smaller than 15");
          // Nested - if statement
          // Will only be executed if statement above  it is true
          if (i < 12)
             System.out.println("i is smaller than 12 too");
     }
     else
          System.out.println("i is greater than 15");
 }
```

```
Enter an ineger: 10
i is smaller than 15
i is smaller than 12 too
```

```
Enter an ineger: 25
```



No output? Why?

Conditional operator

Conditional operator (?:) : shorthand if…else. Ternary operator (takes three operands)

- Operands and ?: form a conditional expression
- Operand to the left of the ? is a **boolean** expression—evaluates to a boolean value (true or false)
- Second operand (between the ? and :) is the value if the boolean expression is true
- Third operand (to the right of the :) is the value if the boolean expression evaluates to false.

Conditional operator (cont.)

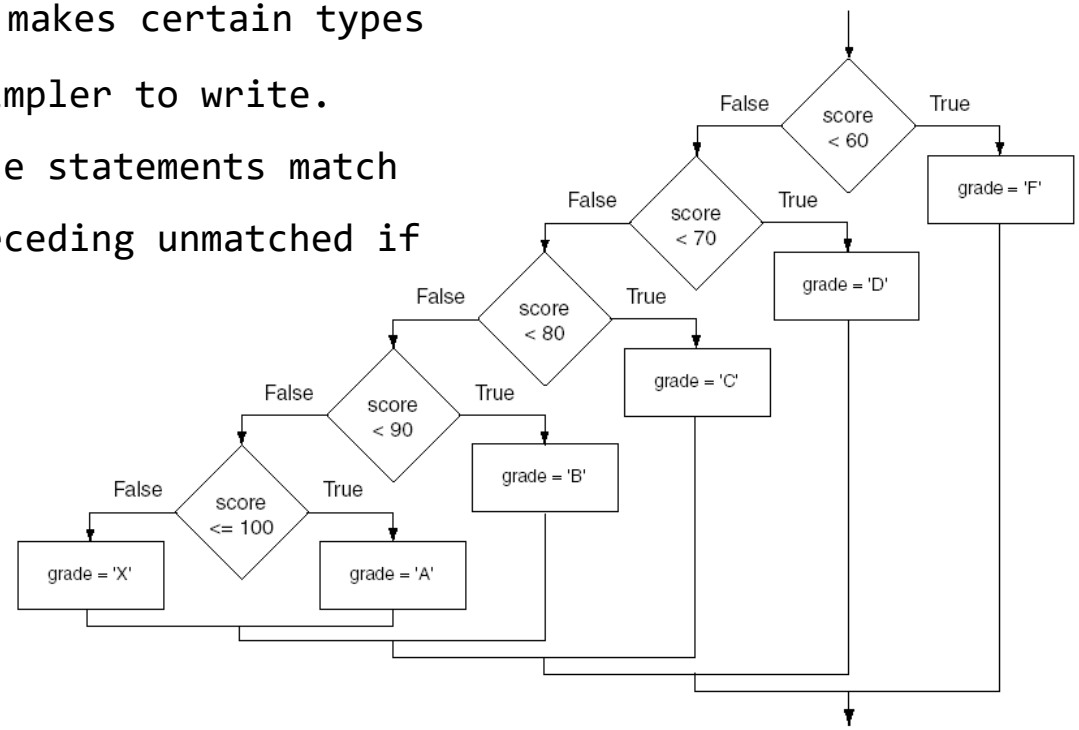
```
public static void main(String[] args) {
    double studentGrade =70;
    double gpa =2.5;
    boolean pass= studentGrade >= 60? true : false;
    boolean isHonor = gpa >= 3.5 ? true : false;
    System.out.println("Pass? " + pass);
    System.out.println("Honor? " + isHonor);
    //Another way
    System.out.println(gpa>=3.5? " Dean List " : "Not in the List");

}
```

```
Pass? true
Honor? false
Not in the List
```

if…else ..if Statement

- **Nested if** statements can become very complex.
- The **if-else-if** statement makes certain types of nested decision logic simpler to write.
- Care must be used since else statements match up with the immediately preceding unmatched if statement.



if…else if Statement

```
public static void main(String[] args) {
     int testScore;
     Scanner input = new Scanner(System.in);
     System.out.print( "Enter your test score (0-100):");
     testScore = input.nextInt();
      // Display the grade.
      if (testScore < 60)
          System.out.println( "Your grade is F.");
      else if (testScore < 70)
          System.out.println( "Your grade is D.");
      else if (testScore < 80)
          System.out.println( "Your grade is C.");
      else if (testScore < 90)
          System.out.println( "Your grade is B.");
      else if (testScore <= 100)
          System.out.println( "Your grade is A.");
      else                    // Invalid score
          System.out.println( "Your score is invlaid.");
      input.close();
}
```

```
Enter your test score (0-100):77
Your grade is C.
```

Logical Operators

- Java provides **two binary** logical operators **(&& and ||)** that are used to combine boolean expressions.
- Java also provides one **unary (!)** logical operator to reverse the truth of a boolean expression.

Logical OR

Expression 1	Expression 2	Expression1 || Expression2
true	false	true
false	true	true
false	false	false
true	true	true

Logical AND

Expression 1	Expression 2	Expression1 && Expression2
true	false	false
false	true	false
false	false	false
true	true	true

Logical NOT

Expression 1	!Expression1
true	false
false	true

Logical Operators

```java
 // initializing variables
int a = 10, b = 20, c = 20, d = 0;
// Displaying a, b, c
System.out.println("Var1 = " + a);
System.out.println("Var2 = " + b);
System.out.println("Var3 = " + c);

// logical AND to verify two constraints
if ((a < b) && (b == c)) {
    d = a + b + c;
    System.out.println("Both conditions are true. The sum is: " + d);
}
else
    System.out.println("False conditions");

// using logical OR to verify two constraints
if (a > b || c == d)
    System.out.println("One or both the conditions are true");
else
    System.out.println("Both the conditions are false");

// Using logical NOT operator
System.out.println("!(a < b) = " + !(a < b));
System.out.println("!(a > b) = " + !(a > b));
```

```
Var1 = 10
Var2 = 20
Var3 = 20
Both conditions are true. The sum is: 50
Both the conditions are false
!(a < b) = false
!(a > b) = true
```

Range Check Using if Statement

- If statement with relational operator can be used to check if a value is within a valid range.

```
public static void main(String[] args) {

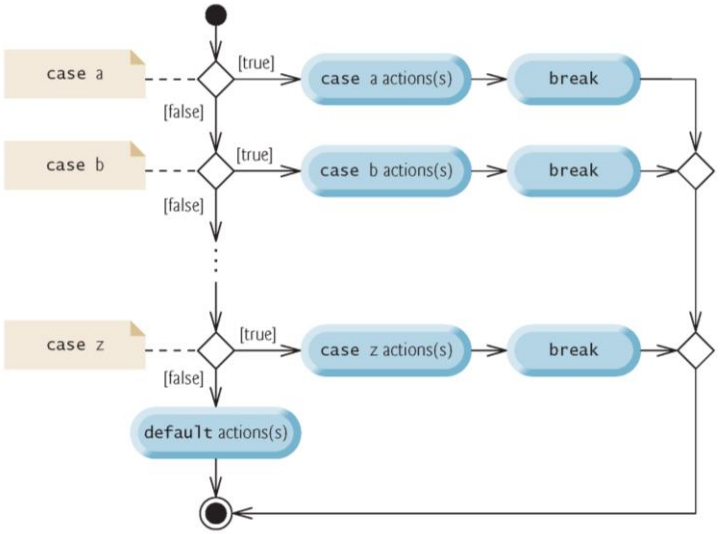
    final double MIN_PH=0;
    final double MAX_PH=14;
    double phValue;
    Scanner input = new Scanner(System.in);
    System.out.print("Enter pH value (0-14) : ");
    phValue = input.nextDouble();
    //Check for range validity
    if (phValue < MIN_PH || phValue > MAX_PH) {
        System.out.println("Invalid pH value...");
    }else if (phValue < 7) {
        System.out.println("Solution is ACIDIC.");
    }else if (phValue==7) {
        System.out.println("Solution is NEUTRAL.");
    }else {
        System.out.println("Solution is BASE.");
    }
    input.close();
}
```

```
Enter pH value (0-14) : 8
Solution is BASE.
```

```
Enter pH value (0-14) : 777
Invalid pH value...
```

The switch Statement

- The if-else statement allows you to make true / false branches.
- The **switch** statement allows you to use an ordinal value to determine how a program will branch.
- The switch statement can evaluate an **integer** type or **character** type variable and make decisions based on the value.
- The **break** and **default** keywords are optional.



The switch Statement

```
public static void main(String[] args) {
    //Declaring a variable for switch expression
    int number;
    Scanner input = new Scanner(System.in);
    System.out.print("Enter an integer: ");
    number = input.nextInt();
    //Switch expression
    switch(number){
        //Case statements
        case 10: System.out.println("10");
                 break;
        case 20: System.out.println("20");
                 break;
        case 30: System.out.println("30");
                 break;
        //Default case statement
        default:System.out.println("Not in 10, 20 or 30");
    }
    input.close();

}
```

```
Enter an integer: 20
20
```

```
Enter an integer: 33
Not in 10, 20 or 30
```

The switch Statement – Fall Through

- The Java switch statement is **fall-through**. It means it executes all statements after the first match if a **break** statement is not present.

```
public static void main(String[] args) {
     int number=20;
        //switch expression with int value
        switch(number){
        //switch cases without break statements
        case 10: System.out.println("10");
        case 20: System.out.println("20");
        case 30: System.out.println("30");
        default:System.out.println("Not in 10, 20 or 30");
        }

}
```

```
20
30
Not in 10, 20 or 30
```

Java Switch Statement with String

- Java allows us to use strings in switch expression since **Java SE 7.** The case statement should be string literal.

```
public static void main(String[] args) {
    //Declaring String variable
    String levelString="Expert";
    int level=0;
    //Using String in Switch expression
    switch(levelString){
    //Using String Literal in Switch case
        case "Beginner": level=1;
                         break;
        case "Intermediate": level=2;
                         break;
        case "Expert": level=3;
                         break;
        default: level=0;
                         break;
    }
    System.out.println("Your Level is: "+level);

}
```

```
Your Level is: 3
```

Java Switch Statement: Empty Case

- Empty case means OR

```
public static void main(String[] args) {
    int day;
    Scanner input = new Scanner(System.in);
    System.out.print("Enter Week Day Number (Monday = 1) : ");
    day = input.nextInt();
    switch (day) {
      case 1:
      case 2:
      case 3:
      case 4:
      case 5: System.out.println("Weekday"); break;
      case 0:
      case 6: System.out.println("Weekend");
    }
    input.close();
}
```

```
Enter Week Day Number (Monday = 1) : 3
Weekday

Enter Week Day Number (Monday = 1) : 6
Weekend

Enter Week Day Number (Monday = 1) : 22
```



No output? Why?